13807 TILLIER Nathan

Calcul de requêtes acycliques conjonctives dans une base de données

Un élément crucial d'une infrastructure informatique est la base de donnée qu'elle interroge grâce à des requêtes. Certaines requêtes sont très complexes, mais il s'avère qu'une partie d'entre elles appartient à une même catégorie, celle des requêtes acycliques conjonctives.

Mon étude s'attache à étudier et implémenter entre autres l'algorithme de Yannakakis, permettant de calculer ces requêtes en temps linéaire. Cet algorithme repose sur le fait que leurs variables ne créent pas de cycle dans l'hypergraphe associé, et s'inscrit donc pleinement dans le thème de l'année.

Le candidat atteste avoir travaillé en monôme.

Positionnement thématique (ÉTAPE 1) :

- *INFORMATIQUE (Informatique pratique)*
- *INFORMATIQUE (Informatique Théorique)*

Mots-clés (ÉTAPE 1) :

Mots-clés (en français)	**Mots-clés (en anglais)**
Base de données	*Database*
Algèbre relationnelle	*Relational algebra*
Hypergraphe	*Hypergraph*
Acyclicité	*Acyclicity*
Algorithmique	*Algorithmics*

Bibliographie commentée

Les bases de données doivent être en capacité de répondre à un nombre important de requêtes par secondes : elles doivent donc être très efficaces lors de leurs calculs. Une bonne partie de ces requêtes appartient à une même catégorie, qu'on appelle requêtes acycliques conjonctives, parmi lesquelles on retrouve les jointures (par exemple JOIN ON en SQL) permettant d'effectuer des recherches mélangeant plusieurs tables. De manière plus générale, de nombreuses requêtes permettant de relier plusieurs tables de données entre elles lors de la recherche sont acycliques conjonctives.

La fondation théorique des bases de données est l'algèbre relationnelle, une théorie introduite par Edgar F. Codd en 1970 [1]. Dans cette dernière, une requête conjonctive est constituée de deux parties : l'en- tête où on indique les valeurs qu'on souhaite obtenir, et le corps de la requête qui explicite les conditions portant sur ces valeurs. Par exemple, si on cherche à déterminer l'ensemble des noms des communes françaises avec le nom du département associé, on liste dans l'en-tête les variables désignant le nom de la commune et celui du département, puis le corps de la requête contiendra la condition associée : il faut que l'identifiant du département corresponde à l'identifiant du département de la commune.

On peut prouver que calculer ce type de requête est NP-difficile [2], et on a donc cherché à déterminer un algorithme légèrement plus restrictif déterminant le résultat d'une telle requête en temps polynomial, voire linéaire. Or il s'avère que certaines requêtes conjonctives, dites acycliques, sont calculables en temps linéaire grâce à l'algorithme de Yannakakis [3]. L'intérêt de cette restriction est qu'elle s'applique dans la pratique à un grand nombre des requêtes effectuées.

Cet algorithme repose sur l'introduction d'un hypergraphe associé à une requête. Les hypergraphes sont essentiellement une généralisation des graphes introduite par Claude Berge [4], dans laquelle une arête peut relier plusieurs sommets. Ce point de vue différent, liant théorie des hypergraphes et algèbre relationnelle, permet de construire un arbre nommé arbre de jointure, à partie duquel on peut construire le résultat en trois parcours : deux de pré-calcul en partant des feuilles jusqu'à la racine de l'arbre puis de la racine de l'arbre jusqu'aux feuilles, et un dernier pour récupérer le résultat [5].

Toutefois, à cause de ces trois parcours différents de l'arbre de jointure, la complexité linéaire possède un facteur constant non négligeable en pratique. Une solution possible est par exemple de construire le résultat en deux parcours seulement. De plus, certains documents récents suggèrent certaines modifications à apporter à la construction de l'arbre de jointure, ainsi qu'au parcours de cet arbre, permettant de tirer profit de l'exécution en parallèle [6]

Problématique retenue

Comment peut-on exécuter des requêtes en temps linéaire sur une base de données ?

Objectifs du TIPE du candidat

Il s'agira dans un premier temps d'étudier les cas d'usage et la formalisation de ce type de requêtes en algèbre relationnelle, puis les algorithmes utilisés pour

- détecter les requêtes appartenant à cette catégorie (réduction Graham-Yu-Ozsoyoglu)

- les exécuter de manière efficace sur une instance (algorithme de Yannakakis)

Ensuite, on implémentera une structure sous-jacente de base de données, puis les algorithmes abordés pour résoudre les requêtes acycliques conjonctives. On discutera des résultats expérimentaux (taille de l'instance, temps de pré-calcul, temps d'exécution total) sur différents exemples.

Enfin, on abordera les modifications possibles de l'algorithme afin de l'optimiser dans la pratique.

Références bibliographiques (ÉTAPE 1)

[1] CODD E. F. : A relational model of data for large shared data banks. *: https://doi.org/10.1145/362384.362685*

[2] KOUTRIS P. : Foundations of Data Management : Acyclic Conjunctive Queries *: https://pages.cs.wisc.edu/~paris/cs784-f19/lectures/lecture2.pdf*

[3] YANNAKAKIS M. : Algorithms for Acyclic Database Schemes *: Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7, (1981)*

[4] BERGE C. : Graphes et hypergraphes *: Graphes et hypergraphes Dunod (1970)*

[5] HU X. : Algorithmic Aspects of Query Processing - Traditional Query Processing *: https://cs.uwaterloo.ca/~xiaohu/courses/CS848/CS848-F25-03-Yannakakis-part2.pdf*

[6] LUO Z., BROECK W. V. DEN, BROECK G. V. DEN, WANG Y. R. : Algorithms for Optimizing Acyclic Queries *: https://arxiv.org/abs/2509.14144*

[7] WOLFGANG GATTERBAUER : Toward Responsive DBMS : Part 3 Acyclic Queries & Enumeration *: https://www.youtube.com/watch?v=toi7ysuyRkw*